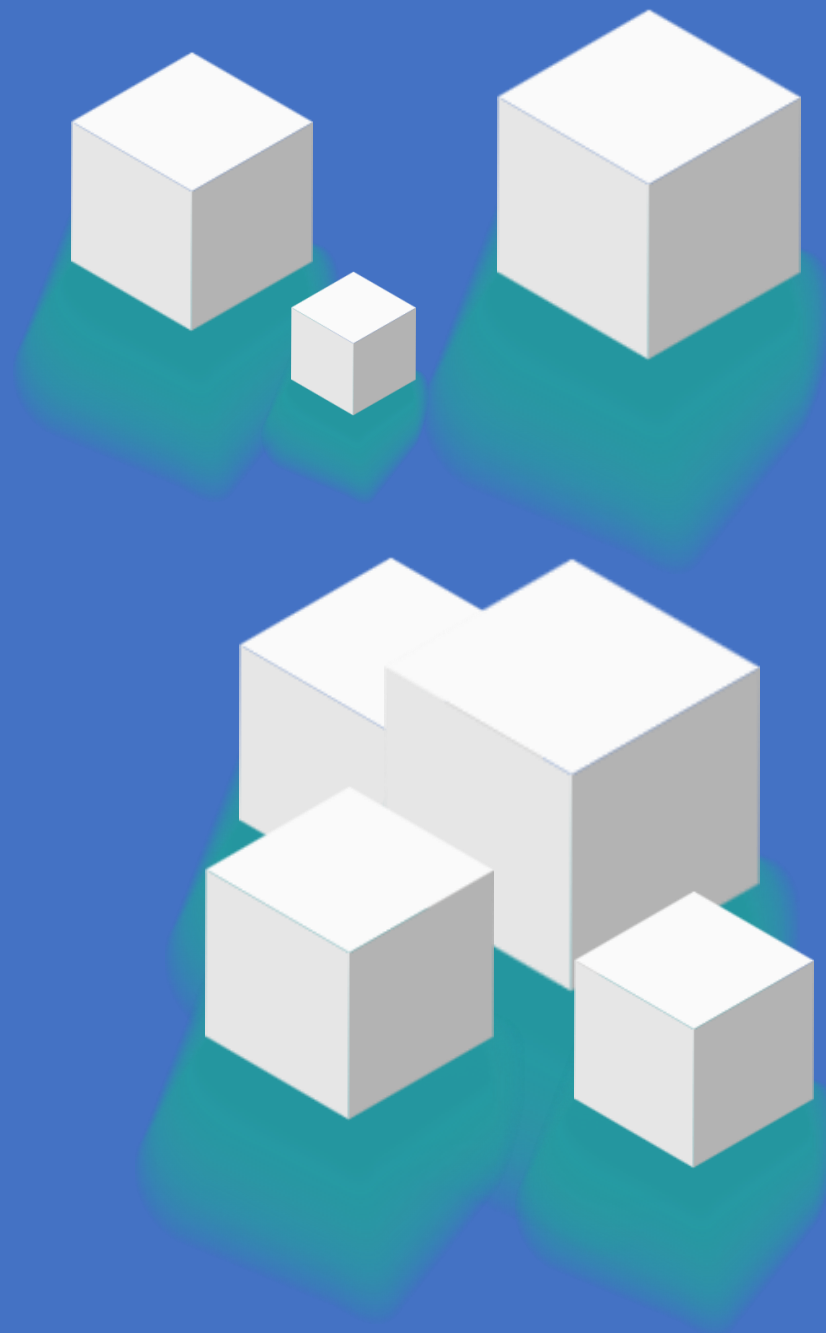
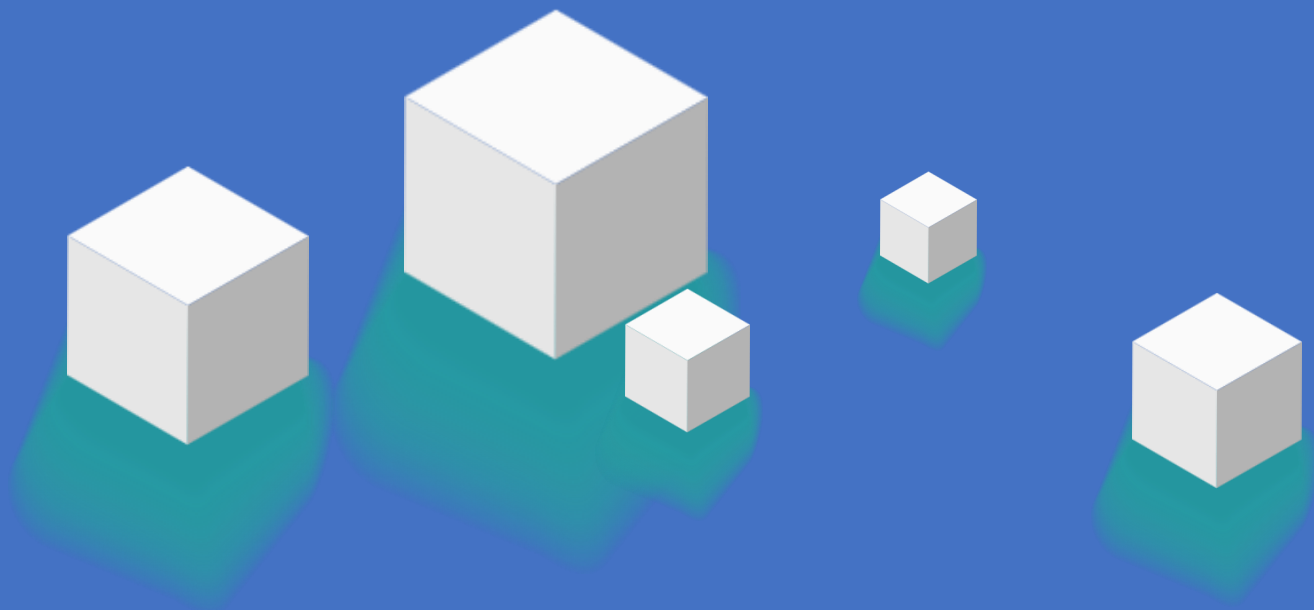


团队复盘与持续进化



>> 今天的学习目标

团队复盘与持续进化

- BHB 归因模型
- CASE：实盘交割单归因分析
- 基于Brinson归因的应用体系
- CASE：模拟盘持仓Brinson分析
- CASE：Walk-Forward Analysis (前进式验证)
- CASE：策略生命周期

持续进化

Thinking: 我们是否需要持续进化的机制?

- 赚了是运气还是实力? => Brinson 归因 (CASE-A)
- 策略真的有效吗 还是过拟合? => Walk-Forward 过拟合检测 (CASE-B)
- 多个策略怎么管理? => 策略生命周期 (CASE-C)

没有持续进化机制的策略, 90% 会在 3-6 个月内失效。

量化的本质是统计套利, 一旦市场风格变化, 你的 alpha 就消失了。

BHB 归因模型

BHB 归因模型

Brinson-Hood-Beebower (BHB) 归因模型

1986 年 BHB 三人提出, 把组合相对基准的超额收益拆解成 3 部分:

超额收益 = 配置效应AE + 选择效应SE + 交互效应IE

- $AE = \sum (W_p - W_b) \times R_b \leftarrow$ 行业配置贡献 => 超配/低配了哪些行业, 以及这些行业在基准中的表现
- $SE = \sum W_b \times (R_p - R_b) \leftarrow$ 个股选择贡献 => 在基准权重框架下, 选股能力带来的超额收益
- $IE = \sum (W_p - W_b) \times (R_p - R_b) \leftarrow$ 交互效应 => 配置与选股的交叉项: 既超配了, 又恰好选到了跑赢基准的股票

W_p 组合在某行业的权重

W_b 基准 (沪深 300) 在该行业的权重

R_p 组合在该行业持仓的加权收益

R_b 基准在该行业的收益

$W = \text{Weight}$ (权重)

$R = \text{Return}$ (收益率)

$p = \text{portfolio}$ (投资组合, 即你的组合)

$b = \text{benchmark}$ (基准, 即沪深300)

BHB 归因模型

Thinking: BHB归因分析是如何计算的?

假设你管理一只基金，基准是沪深300。行业假设只有：科技、消费、其他

行业	组合权重 Wp	基准权重 Wb	组合收益 Rp	基准收益 Rb
科技	40%	30%	15%	10%
消费	30%	40%	8%	5%
其他	30%	30%	5%	5%

Step1, 先算总收益:

- 组合收益 = $40\% \times 15\% + 30\% \times 8\% + 30\% \times 5\% = 9.9\%$
- 基准收益 = $30\% \times 10\% + 40\% \times 5\% + 30\% \times 5\% = 6.5\%$
- 超额收益 = $9.9\% - 6.5\% = 3.4\%$

BHB 归因模型

Step2, 分步拆解

1) 配置效应 AE (赌赛道)

$$AE = \sum (W_p - W_b) \times R_b$$

- 科技: $(40\% - 30\%) \times 10\% = +1.0\%$

(你超配了10%科技, 科技基准涨10%, 这10%超配帮你赚了1%)

- 消费: $(30\% - 40\%) \times 5\% = -0.5\%$

(你低配了10%消费, 消费基准涨5%, 这10%低配让你少赚0.5%)

- 其他: $0 \times 5\% = 0$

$$AE = +0.5\%$$

=> 整体超配了强势行业、低配了弱势行业, 配置贡献正收益。

BHB 归因模型

2) 选择效应 SE (选股Alpha)

$$SE = \sum W_b \times (R_p - R_b)$$

- 科技: $30\% \times (15\% - 10\%) = +1.5\%$

(在科技行业里, 你选的股比行业基准多涨了5%, 按基准权重30%算贡献1.5%)

- 消费: $40\% \times (8\% - 5\%) = +1.2\%$

- 其他: $30\% \times 0 = 0$

$$SE = +2.7\%$$

=> 在两个行业里都跑赢了行业指数, 选股能力强。

BHB 归因模型

3) 交互效应 IE (双重加持)

$$IE = \sum (W_p - W_b) \times (R_p - R_b)$$

- 科技: $(40\% - 30\%) \times (15\% - 10\%) = +0.5\%$

(你不仅超配科技, 且科技里选股还跑赢, 双重正向叠加)

- 消费: $(30\% - 40\%) \times (8\% - 5\%) = -0.3\%$

(你低配消费, 但消费里选股跑赢)

- 其他: 0

$$IE = +0.2\%$$

BHB 归因模型

Step3, 汇总结果

归因项	金额	占比	解读
超额收益	3.4%	100%	你总共跑赢基准3.4个百分点
配置效应 AE	+0.5%	14.7%	你赌对了赛道（超配科技）
选择效应 SE	+2.7%	79.4%	主要靠选股实力
交互效应 IE	+0.2%	5.9%	超配+选对的额外加成

CASE：实盘交割单归因分析

TO DO：找到你的实盘交割单，并进行归因分析

方式 1: 从券商导出的"历史成交"csv 加载 (推荐, 真实手续费, 含港股)

```
df = load_from_csv("data/历史成交_cy_260101-260325.csv")
```

方式 2: 从模拟盘 / 实盘的 live_state.json 加载

```
df = load_from_state_json("path/to/live_state.json")
```

```
df = add_costs(df) # 模拟盘没有真实手续费, 用成本模型估算
```

方式 3: 从 miniQMT 实时拉 (真实实盘后用)

```
df = load_from_miniqmt(qmt_path, account_id)
```

CASE：实盘交割单归因分析

登录 PC 版客户端

顶部导航：账户 → 普通交易 → 历史成交

选择起止日期（单次区间通常不超过约 90 天，可多次导出）

下载 CSV；多个文件放在同一工作目录，脚本可自动合并



CASE：实盘交割单归因分析

< 返回

🏠 首页 / 普通交易 / 历史成交

起始日期

2026-01-01

终止日期

2026-03-25

查询

导出

筛选

汇总

资金账号:

序号	证券名称	证券代码	成交日期	成交时间	买卖方向	成交数量	成交价格	成交金额	佣金	净佣金	过户费	印花税	结算费	成交编号	股东代码	备注
1	九安医疗	002432	2026-03-06	09:31:00	买入	3000	49.51	148530	22.28	14.24	0	0	0	4214363	0331641444	九安医疗证券买入
2	新天然气	603393	2026-03-04	14:20:00	卖出	2000	40.8	81600	12.24	7.83	0.81	40.8	0	66354820	A484381710	新天然气证券卖出
3	联合能源集团	00467	2026-03-03	--	卖出	50000	0.96	42249.6	25.35	--	0	42.2496	0	467000006405	A484381710	港股待交收卖出 成交00467价格 0.96港元数量 50000.00
4	联合能源集团	00467	2026-03-03	--	买入	50000	0.93	40938.6	24.57	--	0	41.3788	0	4670000005583	A484381710	港股待交收买入 成交00467价格 0.93港元数量 50000.00
5	新天然气	603393	2026-03-03	10:19:00	买入	2000	40.85	81700	12.26	7.84	0.82	0	0	36544608	A484381710	新天然气证券买入
6	贵州茅台	600519	2026-02-24	14:15:00	卖出	100	1466	146600	21.99	14.06	1.47	73.3	0	51331567	A484381710	贵州茅台证券卖出
7	纳指ETF	513100	2026-01-20	14:55:00	买入	100	1.876	187.6	5	4.99	0	0	0	75717286	A484381710	纳指ETF证券买入
8	贵州茅台	600519	2026-01-20	09:25:00	买入	100	1379.5	137950	18.68	11.22	1.38	0	0	302563	A484381710	贵州茅台证券买入

文件名常见形式为 历史成交_账号_日期范围.csv

共有8条

<1>

50 条/页

CASE：实盘交割单归因分析

CSV 核心字段（金阳光格式）

字段	说明	示例
证券名称	股票名称	贵州茅台
证券代码	6 位代码	600519
成交日期	交易日期	2026-02-24
买卖方向	买入 / 卖出	卖出
成交数量	股数	100
成交价格	成交均价	1466.00
成交金额	总金额	146600.00
佣金	券商佣金	21.99
印花税	卖出侧常见	73.30
过户费	沪市等	1.47

CASE：实盘交割单归因分析

#加载真实交割单 8 笔

python attribution/trade_record.py --source csv

[原始交易记录] 8 笔 来源: CSV (历史成交_cy_260101-260325.csv)

trade_date	code	name	side	quantity	price	amount	commission	stamp_duty	
2026-01-20	600519.SH	贵州茅台	buy	100	1379.500	137950.0	18.68	0.0000	
2026-01-20	513100.SH	纳指ETF	buy	100	1.876	187.6	5.00	0.0000	
2026-02-24	600519.SH	贵州茅台	sell	100	1466.000	146600.0	21.99	73.3000	
2026-03-03	603393.SH	新天然气	buy	2000	40.850	81700.0	12.26	0.0000	
2026-03-03	00467.HK	联合能源集团	sell	50000	0.960	42249.6	25.35	42.2496	← 港股
2026-03-03	00467.HK	联合能源集团	buy	50000	0.930	40938.6	24.57	41.3788	← 港股 T+0
2026-03-04	603393.SH	新天然气	sell	2000	40.800	81600.0	12.24	40.8000	
2026-03-06	002432.SZ	九安医疗	buy	3000	49.510	148530.0	22.28	0.0000	

CASE：实盘交割单归因分析

[成本汇总]

佣金合计：142.37 元

印花税合计：197.73 元

过户费合计：4.48 元

总成本：344.58 元

成交金额：679,755.80 元

总成本/成交：5.1 bps ← 真实账户的成本水位

数据看出来的几件事：

- **5.1 bps 成本**：A 股佣金 ~1.5 bps + 卖出印花税 5 bps + 港股佣金更高, 平均下来 ~5 bps.

高频策略要小心 => 如果 $\alpha < 10 \text{ bps}$ => 直接被成本吃掉

- **港股 T+0**：联合能源 50000 股, 同一天 sell + buy, 这是港股 T+0 + 双向佣金的痕迹
- **持仓周期可见**：茅台 1-20 买 → 2-24 卖 (35 天), 新天然气 3-3 买 → 3-4 卖 (1 天)

bps = basis points (基点) , 1 bps = 0.01% (万分之一) , 100 bps = 1%。

Brinson 归因

Brinson 归因：三因子拆解

```
def brinson_attribution(portfolio_weights, benchmark_weights,
                        portfolio_returns, benchmark_returns):
    for ind in industries:
        wp = portfolio_weights[ind] # 组合权重
        wb = benchmark_weights[ind] # 基准权重
        rp = portfolio_returns[ind] # 组合在该行业的收益
        rb = benchmark_returns[ind] # 基准在该行业的收益

        # BHB 三因子拆解
        ae = (wp - wb) * rb          # 配置效应: 超配/低配 × 该行业基准收益
        se = wb * (rp - rb)          # 选股效应: 基准权重 × 选股 alpha
        ie = (wp - wb) * (rp - rb)   # 交互: 双重加持
```

Brinson 归因 (csv1)

运行: CSV1 (1-3 月, 8 笔) 真实交割单 Brinson

cd CASE-AI量化系统 && python app.py

=> http://127.0.0.1:7865/review -> Brinson 归因 -> 数据源 选择 历史成交_cy_260101-260325.csv

1) 单股 PnL 拆解 (Profit and Loss 盈亏 / 损益)

代码	名称	申万一级	买入	卖出	净仓	总损益	收益率
002432.SZ	九安医疗	医药生物	148,530	0	3,000	-1,410	-0.95% (期末浮亏)
00467.HK	联合能源	境外/其他	40,939	42,250	0	+1,311	+3.20% (港股 T+0)
513100.SH	纳指 ETF	境外/其他	188	0	100	-7	-4.00% (100 股小试)
600519.SH	贵州茅台	食品饮料	137,950	146,600	0	+8,650	+6.27% 大赢家
603393.SH	新天然气	公用事业	81,700	81,600	0	-100	-0.12% (平进平出)

Brinson 归因 (csv1)

2) 行业聚合 + Brinson 三因子 (vs 沪深300 同区间)

行业	Wp (投入占比)	Rp (买入金额加权)	注
食品饮料	33.70%	+6.27%	茅台一战定胜负
医药生物	36.29%	-0.95%	九安医疗仍持仓 mark-to-market
公用事业	19.96%	-0.12%	新天然气基本平进平出
境外/其他	10.05%	+3.17%	港股 + 纳指 ETF 混合

组合区间收益: +2.06% (8,444 / 409,306)

基准 (沪深300): +0.26%

超额: +1.80%

配置效应 (赛道): -3.38%

选股效应 (Alpha): -0.09%

交互效应: +5.27% <- 几乎全部超额来源

Brinson 归因 (csv1)

Wp 投入占比计算:

$Wp(\text{行业}) = \text{该行业总买入金额} / \text{组合总买入金额}$

行业	该行业买入	/ 总买入	= Wp
食品饮料	137,950	/ 409,307	= 33.70%
医药生物	148,530	/ 409,307	= 36.29%
公用事业	81,700	/ 409,307	= 19.96%
境外/其他	40,939 + 188 = 41,127	/ 409,307	= 10.05%
			合计 100%

Rp 买入金额加权收益率:

$Rp(\text{行业}) = \text{该行业总损益} / \text{该行业总买入金额}$

行业	该行业损益	/ 该行业买入	= Rp
食品饮料	+8,650	/ 137,950	= +6.27%
医药生物	-1,410	/ 148,530	= -0.95%
公用事业	-100	/ 81,700	= -0.12%
境外/其他	+1,311 + (-7) = +1,304	/ 41,127	= +3.17%

Brinson 归因 (csv2)

运行: CSV2 (3-5 月, 7 笔) 真实交割单

1) 单股 PnL 拆解 (CSV2)

代码	名称	申万一级	买入	卖出	净仓	总损益	收益率
002432.SZ	九安医疗	医药生物	148,530	228,600	0	+80,070	+53.91% 大胜
00467.HK	联合能源	境外/其他	40,939	42,250	0	+1,311	+3.20% (T+0 套)
603393.SH	新天然气	公用事业	81,700	81,600	0	-100	-0.12% (平价)
688525.SH	佰维存储	电子	215,200	0	800	-3,136	-1.46% (新建仓浮亏)

说明: CSV2 的九安医疗是从 CSV1 末持有至 4 月 29 日卖出

1-3 月份-0.95% 浮亏中那段, 在 3 月之后翻身大涨 +53.91%

=> 这是为什么流水口径要 mark-to-market 期末市值, 否则区间 1 那段隐藏的"浮亏 → 翻身"过程就丢了.

流水只记录动作, 盯市还原状态。

Brinson 归因 (csv2)

2) Brinson 三因子 (CSV2)

组合区间收益: +16.07% (78,145 / 486,369)

基准 (沪深300): -2.64% <- 大盘下跌

超额: +18.71%

配置效应 (赛道): +6.27%

选股效应 (Alpha): +6.42%

交互效应: +6.02%

三因子均衡正向, 这才是真 Alpha 信号

Summary

两份并排 => 这是 Brinson 真正的价值

指标	CSV1 (1-3 月)	CSV2 (3-5 月)	谁更值得相信
组合收益	+2.06%	+16.07%	CSV2 大赚
基准 (沪深300)	+0.26%	-2.64%	CSV2 是熊市背景
超额	+1.80%	+18.71%	CSV2 完胜
配置效应	-3.38%	+6.27%	CSV2 赛道选对了
选股效应	-0.09%	+6.42%	CSV2 真有 Alpha
交互效应	+5.27%	+6.02%	都重仓押对, 但 CSV2 不靠它单独发力
三因子分布	1 个独大 (交互), 2 个负	三个均衡正向	CSV2 是更可复制的能力

Summary

基于Brinson 归因的实战可以发现：

1. 拆开来才知道是真本事还是运气

CSV1 与 CSV2 都跑赢沪深 300，但 CSV1 的 +1.80% 超额几乎全部来自交互效应（重仓押对），配置和选股甚至为负，属于不可复制的运气；

CSV2 的 +18.71% 则由配置、选股、交互三因子均衡正向贡献，这才是可复制的系统性能力。

2. 短区间可能错杀长持仓

CSV1 末九安医疗浮亏 -0.95%，到 CSV2 兑现 +53.91%。

=> 这说明短区间的归因会低估尚未兑现的选股能力，需等持有周期完整再下结论。

Summary

3. 熊市跑赢才是真 Alpha

CSV2 期间沪深 300 下跌 -2.64%，组合仍获 +16.07% 绝对收益。

大盘下跌期能跑出超额，比牛市跑赢更能说明问题 => 熊市出 Alpha。

4. BHB 公式有归类偏差

CSV1 中茅台单兵 +6.27%，BHB 将其收益归入交互而非选股，因基准行业本身也在涨。业内会用 Brinson-Fachler (BF) 改进版，将这部分全归为选股。不过三因子的整体分布方向仍具参考价值。

5. Alpha 必须覆盖摩擦成本

CSV1 摩擦成本约 5.1 bps (~344 元)，Alpha +1.80% (~7,300 元)，净 alpha 仍为正；

CSV2 摩擦成本更高，但 18.71% 的 Alpha 完全覆盖。

核心原则：挣到的 Alpha 必须远大于摩擦成本，否则跑赢只是数字游戏。

基于Brinson归因的应用体系

Brinson 归因在业内已形成四层应用体系：

- 学术基石

Brinson-Hood-Beebower (BHB, 1986) 是标准框架，但其 interaction 项容易失真——当组合配置了基准没有的行业时，收益会被错误归类。

Brinson-Fachler (BF, 1985) 改进版解决了这一反直觉，可以用BF归因。

BHB的反直觉

Thinking: 怎么理解BHB 的反直觉?

当组合配置了基准没有的行业（比如沪深300里没有煤炭，但你的组合买了煤炭）时，

BHB 的 Selection 公式是 $W_b \times (R_p - R_b)$ 。因为 $W_b = 0$ ，Selection 永远等于 0。

=> 你在煤炭上赚的所有钱，既不能算选股能力，也不能算配置能力

=> 只能扔进 Interaction（交互项）里。

=> Interaction 变成了垃圾桶，归因结果违背常识。

BF (Brinson-Fachler) 改进

BF (Brinson-Fachler) 改进

BF 做了两个关键改动，把垃圾桶取消：

效应	BHB	BF
配置	$(W_p - W_b) \times R_b$	$(W_p - W_b) \times (R_b - R_{b_total})$
选股	$W_b \times (R_p - R_b)$	$W_p \times (R_p - R_b)$
交互	有	无（并入选股）

- 配置用相对收益：

不看行业涨了多少，看行业跑赢/跑输大盘多少。超配了一个涨得多但不如大盘的行业，BF 会给出负贡献，更符合直觉。

- 选股用组合权重 W_p ：

即使基准没配这个行业 ($W_b=0$)，只要你在里面选股跑赢行业指数，BF 就能识别出来。

BF (Brinson-Fachler) 改进

假设：

- 基准 (沪深300)：新能源 50% (涨 10%)、银行 50% (涨 0%)，总收益 5%
- 组合：新能源 40% (涨 10%)、银行 50% (涨 0%)、煤炭 10% (涨 50%)
- 组合总收益 9%，超额收益 +4%

BHB 归因：

- 新能源：配置 -1%，选股 0%，交互 0%
- 煤炭：配置 0%，选股 0 (因 $W_b=0$)，交互 +5%
- 结果：煤炭赚的 5% 全被扔进交互项 => 看不出是选对了行业还是选对了个股。

BF 归因：

- 新能源：配置 -0.5% (10% 跑输大盘 5%)，选股 0%
- 煤炭：配置 +4.5% (10% 大幅跑赢大盘 45%)，选股 0% (你刚好跟行业指数持平)
- 结果：没有交互项，4% 超额被拆成"新能源配少了"和"煤炭配对了"，归因清晰可解释。

基于Brinson归因的应用体系

- 公募基金

季报、年报的归因报告普遍以 Brinson 为基线，向投资者透明展示超额收益来自行业配置还是个股选择。

- 商业终端

Wind、Bloomberg、FactSet 等主流平台均内置 Brinson 模块，导入持仓与基准后即可一键生成归因报告，大幅降低分析门槛。

- 量化私募

Brinson 只拆到行业与个股层面，私募通常再叠一层 Barra 风格因子归因，将超额进一步拆分为 Size、Momentum、Value、Beta 等风格暴露与残差 Alpha。

=> 目的是回答核心问题：你赚的钱到底靠小市值运气，还是真 Alpha。

CASE：模拟盘持仓Brinson分析

TO DO：模拟盘持仓Brinson分析

2026-04-01 用100 万资金分散买入 5 只代表性标的, 跑 5 周到 2026-05-06, 看组合相对沪深 300 跑得怎样, 超额从哪里来?

- **持仓:** 600519.SH 贵州茅台 / 688981.SH 中芯国际 / 300750.SZ 宁德时代 / 513100.SH 纳指 ETF / 000001.SZ 平安银行
- **区间:** 2026-04-01 (起始日开盘价) ~ 2026-05-06 (结束日收盘价), ~5 周
- **基准:** 沪深 300 当期成分股 (300 只), 行业内等权 (简化口径, 不按市值)
- **行业分类:** 申万一级 (37 个一级行业, 通过 xtdata 拉取 + 7 天缓存)

代码核心

```
# ---- 步骤 1: 申万一级行业字典 (xtdata + 7 天 cache) ----
def build_industry_map(force_refresh=False) -> dict:
    if not force_refresh:
        cache = _load_industry_cache() # outputs/sw1_industry_map.json
        if cache: return cache

    from xtquant import xtdata
    xtdata.connect()

    sw1_sectors = [s for s in xtdata.get_sector_list()
                    if s.startswith("SW1") and not s.endswith("加权")]

    code_to_ind = {}
    for sector in sw1_sectors:          # SW1电子, SW1食品饮料, ...
        ind_short = sector.replace("SW1", "", 1) # SW1电子 -> 电子
        for code in xtdata.get_stock_list_in_sector(sector):
            code_to_ind[code] = ind_short      # 反向映射
```

```
hs300 = xtdata.get_stock_list_in_sector("沪深300")
return {"code_to_ind": code_to_ind, "hs300": hs300, ...}

# ---- 步骤 2: 单只股票区间收益 (MySQL 日 K) ----
def _stock_return(code, start, end) -> float:
    df = load_daily_kline(code, start, end)      # MySQL 优先, 回退 xtdata
    return float(df["close"].iloc[-1]) / float(df["open"].iloc[0]) - 1.0

# ---- 步骤 3: 组合侧 (Wp + Rp) ----
def _portfolio_side(positions, code_to_ind, start, end):
    rows = []
    for p in positions:
        mv = p["volume"] * p["cur_price"]
        ret = _stock_return(p["code"], start, end)
        ind = code_to_ind.get(p["code"], "境外/其他")
        rows.append({"industry": ind, "mv": mv, "ret": ret})
```

代码核心

```
total_mv = sum(r["mv"] for r in rows)
# 行业聚合: Wp = sum(mv) / total; Rp = mv-weighted ret
Wp = {ind: ... for ind in by_ind}
Rp = {ind: ... for ind in by_ind}
return Wp, Rp, rows

# ---- 步骤 4: 基准侧 (Wb + Rb 等权) ----
def _benchmark_side(hs300_codes, code_to_ind, start, end):
    by_ind = {}
    for code in hs300_codes:
        ret = _stock_return(code, start, end)
        if ret is None: continue      # 停牌 / 退市 剔除
        ind = code_to_ind.get(code, "境外/其他")
        by_ind.setdefault(ind, []).append(ret)
```

```
Wb = {ind: len(rets) / total_valid for ind, rets in by_ind.items()}
Rb = {ind: sum(rets) / len(rets) for ind, rets in by_ind.items()}
return Wb, Rb

# ---- 步骤 5: 调核心 brinson_attribution() ----
def compute_real_brinson(start, end, source="sim"):
    ind_map = build_industry_map()
    positions = _load_sim_positions()      # live_state.json
    Wp, Rp, _ = _portfolio_side(positions, ind_map["code_to_ind"], start,
end)
    Wb, Rb = _benchmark_side(ind_map["hs300"],
ind_map["code_to_ind"], start, end)
    return brinson_attribution(Wp, Wb, Rp, Rb)
```

CASE：模拟盘持仓Brinson分析

运行结果 (2026-04-01 ~ 2026-05-06, 真实快照)

总览：跑赢沪深 300 共 +865 bp (~5 周 / 24 个交易日)

指标	数值	说明
组合收益 R_p	+13.70%	5 只持仓市值加权
基准收益 R_b	+5.05%	沪深 300 等权 + 申万一级聚合
超额 $R_p - R_b$	+8.65%	=配置 + 选股 + 交互
配置效应 (赌赛道)	+2.16%	电子/电力设备 超配获利
选股效应 (Alpha)	-0.81%	茅台单兵拖累, 其它 3 只其实跑赢板块
交互效应	+7.30%	异常高 -- 主要是纳指 ETF (+3.81%) 的 BHB反直觉

CASE：模拟盘持仓Brinson分析

行业贡献排序 (top 5 + bottom 3):

行业	组合 W_p	基准 W_b	组合 R_p	基准 R_b	配置	选股	交互	合计
电子 (中芯国际)	24.6%	11.7%	+31.72%	+29.03%	+3.77%	+0.31%	+0.35%	+4.43%
境外/其他 (纳指ETF)	22.6%	0.0%	+16.86%	0.00%	0	0	+3.81%	+3.81%
电力设备 (宁德时代)	19.0%	4.7%	+13.48%	+4.33%	+0.62%	+0.43%	+1.31%	+2.36%
银行 (平安银行)	19.8%	8.0%	+2.07%	-1.52%	-0.18%	+0.29%	+0.42%	+0.53%
汽车 (低配跌的)	0%	3.0%	0	-6.97%	+0.21%	+0.21%	-0.21%	+0.21%
食品饮料 (贵州茅台)	14.0%	4.0%	-6.38%	-3.35%	-0.33%	-0.12%	-0.30%	-0.76%
家用电器 (低配涨的)	0%	5.3%	0	+8.91%	-0.48%	-0.48%	+0.48%	-0.48%
通信 (低配涨的)	0%	3.3%	0	+12.55%	-0.42%	-0.42%	+0.42%	-0.42%

说明：低配跌的行业是给组合带来正贡献的机会规避 (基准跌了你没持, 相对就跑赢了);

低配涨的是负贡献的踏空成本 (基准涨了你没持, 相对就跑输了).

CASE：模拟盘持仓Brinson分析

Thinking: Brinson归因分析的价值是什么?

洞察 1: 纳指 ETF 贡献 +3.81%, 但 Brinson 把它归到交互效应

这是 Brinson BHB 公式的经典 quirk (反直觉): 当组合持有基准没有的行业 ($W_b=0$), 公式自动崩塌:

$$\text{配置效应} = (W_p - W_b) \times R_b = 0.226 \times 0 = 0$$

$$\text{选股效应} = W_b \times (R_p - R_b) = 0 \times \dots = 0$$

$$\text{交互效应} = (W_p - W_b) \times (R_p - R_b) = 0.226 \times 0.1686 = +0.0381 \leftarrow \text{全跑这里}$$

这就是为什么 interaction 高达 +7.30%, 其中 +3.81% 是这一项.

解决方法: Brinson-Fachler (BF) 改进版: 把这部分计入配置效应, interaction 项消失

多基准: 纳指 ETF 用海外指数单独配对基准, 不强行塞进 A 股归因

CASE: 模拟盘持仓Brinson分析

洞察 2: 选股效应 -0.81% 是被茅台一个人拖出来的

逐只对照个股 R_p vs 同行业基准 R_b :

- 茅台 -6.38% vs 食品饮料板块 -3.35% -- 跑输 3.03 pp ⚠️ 唯一拖累项
- 中芯 $+31.72\%$ vs 电子板块 $+29.03\%$ -- 跑赢 2.69 pp ✓ alpha 出现
- 宁德 $+13.48\%$ vs 电力设备 $+4.33\%$ -- 跑赢 9.15 pp ✓ 强 alpha
- 平安 $+2.07\%$ vs 银行 -1.52% -- 跑赢 3.59 pp ✓ alpha 出现

结论: 4 只 A 股里 3 只都跑赢板块, 选股能力其实有, 但茅台权重 14% 的负 alpha (-3 pp) 拖累过大, 把整体 selection 拉到了 -0.81% .

=> 换掉茅台

CASE: 模拟盘持仓Brinson分析

洞察 3: 这是一份"赛道 + 选股双驱动"组合, 但赛道贡献更大

- 超额 +8.65% 中:

- 1) 赛道押注: 配置 +2.16% + 纳指 ETF 的 +3.81% = +5.97% (~69%)
- 2) 选股 alpha: 选股 -0.81% + 中芯/宁德/平安 在 interaction 里的部分 $\approx$ +1.5% (~17%)
- 3) 其他 interaction (含纳指 ETF): 剩余 ~14%

优化方向:

- ① 处置茅台;
- ② 维持电子/电力设备的超配 (有持续 alpha);
- ③ 适度补一些低配但持续涨的板块 (家电/通信), 减少踏空成本

CASE：模拟盘持仓Brinson分析

洞察 4：等权基准会高估超额

沪深 300 实际是市值加权, 茅台 / 工商银行 / 宁德 这些大票占基准的实际权重远高于行业内等权算出来的.

这里为了简化, 用了等权 => 会给小盘股的权重较高, 在牛市小盘行情下会高估基准, 在熊市会低估基准.

真实超额数字可能比 +8.65% 小一些

升级到市值加权需要 Wind / 同花顺 / Choice 当期权重接口.

CASE：模拟盘持仓Brinson分析

洞察 5：同一个组合, 不同观察期, 归因结论可能反转

对照 早期3周(2026-04-01 ~ 04-20) vs 当前5周 (2026-04-01 ~ 05-06):

指标	3 周	5 周	变化
组合收益	+5.36%	+13.70%	+8.34 pp
基准收益	+3.10%	+5.05%	+1.95 pp
超额	+2.26%	+8.65%	+6.39 pp
选股效应	-1.61%	-0.81%	+0.80 pp (改善但仍负)
中芯 vs 电子	跑输 3.17 pp	跑赢 2.69 pp	反转

单次归因的选股能力判断样本量太小, 3 周看到的个股都不行在 5 周就反转了.

=> 任何运气 vs 实力的判断都要做多窗口验证, 单点结论不一定稳定

Brinson归因的工程实现

工程实现复用程度极高, 一份基准侧代码两个口径共享:

lib/brinson_real.py

- └— compute_real_brinson(start, end, source='sim'/'real') # 持仓快照口径
- └— compute_brinson_from_trades(csv_path, start, end) # 交割单流水口径
- └— _portfolio_side(...) # 持仓快照 -> Wp/Rp
- └— _portfolio_side_from_trades(...) # 交割单流水 -> Wp/Rp
- └— _benchmark_side(...) # 沪深300 等权 (两个主入口共用)
- └— _stock_return / _close_at(...) # 区间收益 / 期末收盘价 (mark-to-market)
- └— build_industry_map(...) # 申万一级 cache (两个主入口共用)

attribution/trade_record.py

- └— load_from_csv(csv_path) # 解析中文表头 + gbk 编码 + 港股代码格式化
- └— list_available_csvs() # 扫 data/ 下"历史成交_*.csv" (前端下拉用)
- └— DEFAULT_CSV_PATH = 自动选最新一份 # 文件名按区间倒序, 默认走最新

打卡：交割单 Brinson 归因



以一份券商导出的真实交割单为例 (建议用自己的交割单替换 data/ 下的 CSV), 用 BHB 公式把超额收益拆成 配置 + 选股 + 交互 三块, 看是哪个能力让你跑赢:

- 导出交割单: 任意券商客户端 → 历史成交 → 导出 CSV, 放到 data/ 目录, 文件名带区间即可。
- 加载 + 标准化: 自动识别 中文表头 + 港股代码, 把每一笔成交读成统一格式。
- 算摩擦成本: 按 A 股规则计算佣金 + 印花税 + 过户费, 给出 bps 总成本, 跟 Alpha 对比看是不是真挣到了钱。
- 三因子拆解: 用 BHB 公式拆超额 = 配置效应 + 选股效应 + 交互效应, 基准用沪深 300 同区间。
- 申万行业映射: 用 xtdat 拉成分股 + 行业归属, 写到本地 cache, 后续秒级返回。

Walk-Forward Analysis (前进式验证)

Walk-Forward Analysis (前进式验证)

Thinking: 传统回测的致命缺陷: 用同一份数据找参数 + 评估 $\rightarrow$ 必然过拟合。

如何解决?

Walk-Forward 的解法: 训练和评估完全分离。

时间轴: |---训练 60d---|--评估 20d--|---训练 60d---|--评估 20d--|...

每次训练完后, 用 best_params 在评估期跑

评估期表现 = 真实可达的样本外 (OOS) 表现

Walk-Forward Analysis (前进式验证)

Thinking: 怎么理解Walk-Forward 前进式验证?

传统回测是站着不动——站在 2025 年底，用全年数据一次性选出最优参数，然后幻想这个参数能用到 2026 年。

Walk-Forward 是边走边看：

- 站在 3 月底，用前 3 个月数据训练 → 迈一步到 4~6 月验证
- 整体往前挪 2 个月，站在 5 月底，用前 3 个月重新训练 → 再迈一步到 6~8 月验证
- 重复.....像左脚（训练）落地，右脚（测试）跟上，整个人不断向前移动

Walk-Forward Analysis (前进式验证)

关键指标:

指标	公式	含义
IS Sharpe	训练期 Sharpe 均值	训练时看起来多好
OOS Sharpe	评估期 Sharpe 均值	实盘真实可达
IS/OOS 比例	OOS / IS	1.0 = 完美, < 0.5 = 严重过拟合
过拟合分数	(IS - OOS) / IS	< 0.3 鲁棒, > 0.7 严重过拟合

IS = In-Sample (样本内) , 即训练期的数据。

OOS = Out-of-Sample (样本外) , 即评估期的数据。

$$\text{Sharpe} = \frac{R_p - R_f}{\sigma_p}$$

Rp : 策略日均收益率

Rf: 无风险利率 (通常取 0 或国债收益率, 简化时直接忽略)

σp: 策略日均收益率的标准差

Walk-Forward Analysis (前进式验证)

Thinking: 卖方研究 / 公募 / 量化私募内部如何决定策略能不能上线的?

主要通过下面 3 件事 (按重要性排序):

#	验收手段	解决什么问题	对应实现
1	Walk-Forward + 时间序列 CV	过拟合 (训练评估混用)	walk_forward.py
2	分阶段 OOS (牛 / 熊 / 高低波 分别验证)	策略只在某种市场状态下管用	用回测诊断模块, 改一下日期切片
3	Paper Trading (真金白银前的模拟盘)	滑点、撮合、信号延迟、心理	dry-run 模式, 即模拟盘交易

三件事的顺序:

Walk-Forward (淘汰 80% 烂策略) → 分阶段 OOS (淘汰 80% 运气策略) → Paper Trading (淘汰 80% 工程没就绪策略) → 实盘

经过这三关还能活下来的, 才是 1% 的真策略.

场景说明

Thinking: 我做了 6 个候选参数 (双均线 5/20, 5/30, 10/30, 10/60, 20/60, 20/120), 怎么知道哪个能上实盘?

不能用传统回测! 传统回测的致命问题:

用同一份 2025 全年数据找最优参数 + 用同一份数据评估 = 会过拟合, 实盘会崩

场景说明

Walk-Forward 的具体窗口安排

把数据按下面这种方式切成滚动窗口:

窗口 1: 训练 $[t_0 \sim t_0+120d]$ 找最优参数 (假设 5/20 最优)

评估 $[t_0+120 \sim t_0+180d]$ 用 5/20 看 OOS 表现

窗口 2: 训练 $[t_0+60 \sim t_0+180d]$ 重新找 (可能变 10/30)

评估 $[t_0+180 \sim t_0+240d]$

窗口 3: 训练 $[t_0+120 \sim t_0+240d]$

评估 $[t_0+240 \sim t_0+300d]$

...

最后: 平均所有窗口的 OOS Sharpe = 真实可达的实盘表现

窗口 = 模拟真实交易的时间切片

用过去的数据选参数, 用未来的数据验真假, 而且这个过程要滚动重复多次。

核心代码

```
def walk_forward_analysis(df, strategy_fn, param_grid,
                        train_window=120, test_window=60):
    cursor = 0
    windows = []

    while cursor + train_window + test_window <= len(df):
        train_df = df.iloc[cursor: cursor + train_window]
        test_df = df.iloc[cursor + train_window: cursor + train_window + test_window]
```

1) 在训练集上跑所有参数, 选 (Sharpe, Return) 最好的

```
best_params = None
best_score = (-np.inf, -np.inf)
for params in param_grid:
    trades = strategy_fn(train_df, params)
    sharpe, ret = _sharpe_and_ret(trades, len(train_df))
    if (sharpe, ret) > best_score:
        best_score = (sharpe, ret)
        best_params = params
```

2) 用 best_params 在评估集上跑, 看 OOS

```
oos_trades = strategy_fn(test_df, best_params)
test_sharpe, test_ret = _sharpe_and_ret(oos_trades, len(test_df))

windows.append(WalkForwardWindow(...))
cursor += test_window
```

3) 关键指标: 过拟合分数

```
avg_train = np.mean([w.train_sharpe for w in windows])
avg_test = np.mean([w.test_sharpe for w in windows])
overfit_score = (avg_train - avg_test) / avg_train
```

运行结果

```
python parameter_tuning/walk_forward.py --code 600519.SH --count 800 --train 120 --test 60
```

[各窗口明细]

window	train_period	test_period	best_params	train_sharpe	test_sharpe	
0	2022-12~2023-06	2023-06~2023-09	{ma_short:10, ma_long:60}	0.00	0.00	
1	2023-03~2023-09	2023-09~2023-12	{ma_short:5, ma_long:30}	3.56	0.00	← train 看起来好, test 完全失效
2	2023-06~2023-12	2023-12~2024-03	{ma_short:20, ma_long:120}	0.00	0.00	
3	...					
8	2024-12~2025-06	2025-06~2025-09	{ma_short:5, ma_long:20}	4.95	0.00	← 同样的问题
...						

[汇总]

窗口数 : 11
平均 IS Sharpe : +0.774 ← 训练集看起来很好
平均 OOS Sharpe: +0.000 ← 实盘表现 0
OOS/IS 比例 : 0.00 (1.0 完美, < 0.5 过拟合)
过拟合分数 : +100.00% -> 严重过拟合

[投资建议]

严重过拟合, 不应实盘! 重新设计策略

运行结果

Thinking: 为什么 Sharpe 3.56 实盘 0?

这是过拟合的经典案例:

- 窗口 1 训练: 5/30 双均线 Sharpe 3.56 (看起来牛逼)
- 窗口 1 评估: 同样的 5/30 在下一个 60 天 Sharpe 是 0
- 原因: 训练期市场是某种特定状态 (例: 横盘走 N 字), 5/30 刚好捕捉到了
- 评估期市场状态变了 (例: 单边趋势), 5/30 完全失效

没有 Walk-Forward, 你看到 Sharpe 3.56 如果上了实盘 → 很可能崩。

Walk-Forward 把这个真相提前暴露给你

给生产策略的诊断流程

给生产策略的诊断流程

每周日固定时间跑这个 demo, 检查所有在用策略:

1) 拉本周新增的 K 线

```
xtdata.download_history_data(...)
```

2) 对每个在用策略跑 Walk-Forward

```
for strategy in production_strategies:
```

```
    report = walk_forward_analysis(...)
```

3) 自动告警

```
if report.overfit_score > 0.5:
```

```
    alert_router.alert("WARN",
```

```
        f"策略 {strategy.name} 过拟合分数 {report.overfit_score:.0%}, 建议复盘")
```

4) 触发策略生命周期评估

```
registry.evaluate_and_migrate()
```

Summary

- Walk-Forward（前进式验证）用于识别过拟合：80% 回测漂亮的策略，在滚动样本外测试中都会现形。
- 真正值得参考的不是样本内（IS）Sharpe = 3，而是样本外（OOS）Sharpe 能否稳定为正
=> 实盘只认 OOS。
- 它仅是实盘三道关卡的第一关，后续还需分阶段 OOS 验证与 Paper Trading，层层过滤。
- 一旦发现过拟合，唯一解法是重新设计策略逻辑，而不是微调参数 => 自欺欺人。
- 建议将 Walk-Forward 设为每周自动运行的常规体检，让过拟合预警成为投研流程的常态。

打卡: Walk-Forward 过拟合检测



以双均线策略为例 (示例用茅台, 你换成自己策略 + 自己的标的), 验证参数能不能上实盘:

- 滚动切窗口: 把历史 K 线切成若干个不重叠的训练 + 评估段, 时间方向不回望未来。
- 训练期网格搜索: 在每个训练期把候选参数都跑一遍, 选出 IS Sharpe 最高的 best_params。
- 评估期跑 best_params: 拿训练期选出的参数到下一段评估期跑, 这就是真实可达的 OOS 表现。
- 汇总诊断: IS Sharpe vs OOS Sharpe, 算 $\text{overfit_score} = 1 - \text{OOS}/\text{IS}$, 同时看各窗口最优参数稳不稳。
- 投资建议: < 0.3 可以 dry-run; $0.3 \sim 0.7$ 中度过拟合, 减参数空间 + 加交易成本; > 0.7 必须重新设计策略, 不是换参数。

CASE：策略生命周期

策略生命周期

Thinking: 产品有生命周期, 策略有生命周期吗?

有的, 策略生命周期管理的核心, 是给每套策略一条 KPI 驱动的晋升通道, 而不是凭感觉

借鉴产品生命周期, 策略依次经历五阶段:

孵化 → 模拟 → 试用 → 主力 → 退役

- 升降级完全由客观 KPI 门槛决定: 达标自动晋级, 不达标自动降级或回退
- 资金分配与阶段严格绑定, 遵循"风险递减、信心递增"原则逐级加仓

=> 孵化期零资金纯观察, 模拟期小仓位验证, 试用期逐步放大, 主力期全力运行, 退役期清零退出。

本质是用自动化规则替代主观判断, 让策略在真实场景中靠数据证明自己, 能过才上量。

策略生命周期

Thinking: 我有 10 万本金, 写了一个新策略. 直接 all in 心里没底, 完全不投又怕错过

模拟盘到底跑多久才能上实盘? 上了实盘亏了几天该砍, 还是该忍?

这是每个量化交易者 (散户 / 小私募 / 大机构) 都会遇到的问题. 常见错误做法:

- 凭感觉留着 => 老策略亏了不忍心砍, 占着钱跑了几个月还在亏
- 凭感觉加仓 => 看见某策略 5 天赚了 8% 就 all in, 第 6 天回撤 12% 直接爆仓
- 凭感觉砍掉 => 某策略亏了一周就立刻关掉, 其实只是市场短期噪音

解法: 用客观、KPI 驱动的分阶段加仓 流程, 让数据说话, 不让情绪说话.

核心思想: 新策略不要直接给大钱, 让它先在模拟盘跑, 然后给小钱试, 表现真好了再给大钱; 表现持续变差就停用. 每一步都有客观的 KPI 门槛, 不靠拍脑袋。

阶段生命周期



每一步的升级门槛 (KPI 驱动, 不靠感觉):

迁移	门槛条件	说明
INCUBATING → PAPER	Walk-Forward 评估期 / 训练期 Sharpe 比 > 0.7	参数不是过拟合出来的
PAPER → PROBATION	20+ 个交易日, 累计收益 > 0, 最大回撤 < 5%	模拟盘真的赚钱, 而且不大亏
PROBATION → PRODUCTION	20+ 个交易日, 年化 Sharpe > 0.5	小钱试用 1 个月稳定赚钱
PRODUCTION → RETIRED	连续 14 天 Sharpe < 0.3	主力策略持续两周不行了, 砍
任何阶段 → RETIRED (强制)	30 日最大回撤 > 20%	亏太多, 兜底保命

阶段生命周期

Thinking: 不同阶段的资金占比怎么定 (5% 还是 30%)?

建议数字, 散户可以根据自己的账户大小调. 比如:

- 本金 10 万的: probation 给 5000~1万, production 给 3万
- 本金 100 万的: probation 给 5万, production 给 30 万
- 核心是分阶段, 不是直接 all in. 数字可以自己拍, 但比例感要对

核心代码

```
class StrategyRegistry:
    def evaluate_and_migrate(self) -> List[dict]:
        # 遍历所有策略, 自动决定下一阶段
        migrations = []
        for name, rec in self.strategies.items():
            new_stage = self._decide_next_stage(rec)
            if new_stage and new_stage != rec.stage:
                old = rec.stage
                rec.stage = new_stage
                rec.capital_pct = self._suggest_capital_pct(new_stage)
                rec.capital_allocated = self.total_capital * rec.capital_pct
                rec.history.append({"from": old.value, "to": new_stage.value, ...})
                migrations.append(...)
        return migrations

    def _decide_next_stage(self, rec) -> Optional[LifecycleStage]:
        kpi = rec.kpi
        st = rec.stage
```

```
# 强退役: 30 日回撤 > 20%
if kpi.rolling_30d_maxdd > 0.20:
    return LifecycleStage.RETIRED

if st == LifecycleStage.INCUBATING:
    if kpi.walk_forward_is_oos_ratio >= 0.7:
        return LifecycleStage.PAPER

elif st == LifecycleStage.PROBATION:
    if kpi.days_since_promotion >= 20 and kpi.rolling_30d_sharpe > 0.5:
        return LifecycleStage.PRODUCTION

elif st == LifecycleStage.PRODUCTION:
    if kpi.consecutive_low_sharpe_days >= 14:
        return LifecycleStage.RETIRED

return None # 不变
```

strategy_lifecycle/registry.py

运行结果

即使你只跑 1 个策略, 这套规则也适用. 这里展示一个同时跑 6 个策略的场景

```
python strategy_lifecycle/registry.py
```

注册了 6 个不同阶段的策略:

策略名	当前阶段	KPI 数据
dragon_first_board (龙头首板)	PRODUCTION 主力	Sharpe 1.20, 30d 收益 +4.5%, 回撤 1.8%
multi_factor_csi300 (多因子)	PRODUCTION 主力	Sharpe 0.80, 30d +2.2%, 回撤 3.5%
old_macd_naive (老 MACD)	PRODUCTION 主力	Sharpe 0.15, 连亏 18 天 ← 应该退役
sector_rotation (行业轮动)	PROBATION 试用	Sharpe 0.65, 22 天试用期满
dqn_ethan (强化学习)	PAPER 模拟盘	纸交易 22 天, 收益 +1.2%, 回撤 4%
breakout_naive (突破)	INCUBATING 研发	Walk-Forward IS/OOS 0.78

运行结果

跑完一轮自动评估, 系统给出 4 条迁移建议:

本轮触发 4 个迁移建议:

[sector_rotation] probation -> production

理由: 试用期 Sharpe $0.65 > 0.5$, 升为主力 (资金从 5% 升到 30%)

[dqn_ethan] paper -> probation

理由: 纸交易 22 天, 收益 +1.20%, 回撤 4.00%, 通过试用门槛 (资金从 0 升到 5%)

[breakout_naive] incubating -> paper

理由: Walk-Forward 评估期/训练期 Sharpe 比 $0.78 \geq 0.70$, 通过研发期

[old_macd_naive] production -> retired

理由: 连续 18 天 Sharpe 低于阈值 (资金从 30% 降到 0)

这 4 条迁移如果靠人工拍脑袋, 可能会留住 old_macd_naive (因为是老策略, 有感情) 同时压住 sector_rotation 不敢加仓 (怕新策略不靠谱)

=> 用 KPI 客观判断, 钱自动往真正赚钱的策略流, 不靠感觉.

Summary

策略生命周期管理的真正价值:

- 客观决策: KPI 数字说话, 没有凭感觉
- 资金自适应: 表现好的策略多分钱, 弱的少分或不分
- 自动退役: 失效的老策略不占资金
- 新策略温和上线: 必须经过 4 个阶段才能拿大钱, 防止新欢风险 (5 天赚 8% 就 all in 那种)

这是 Netflix / Spotify 推荐系统的内容生命周期思路, 用在量化策略管理上同样适用。

Summary

Thinking: 机构如何使用策略生命周期?

大型量化机构（基金、券商自营、大型私募）普遍采用策略阶梯机制，叫法各异

——Strategy Pipeline、Paper-to-Prod Promotion、Capital Allocation Tier 等

=> 本质都是分阶段加仓、KPI 驱动升降级。

与理想化全自动模型相比，机构有两点关键差别：

- 一是触发条件更复合。

机构不会只看单一 Sharpe 阈值，而是组合判定：单策略回撤超 8% 即触发强制减仓 50% 的熔断机制，同时监控相对基准的超额收益、换手率异常、波动率突变等指标，形成多维风控网。

- 二是人机协同决策。

系统每日自动计算 KPI 并给出升降级建议，但最终拍板权仍在基金经理。

只有策略数量极多的高频做市商，才会采用完全机器自动决策。

Summary

Thinking: 对散户和小私募的启示是什么?

如果你只有 1-3 个策略, 无需搭建复杂系统, 核心是把分阶段加仓的流程固化下来——

- 新策略绝不 all in, 先用小仓位验证;
- 每一步都预设客观门槛 (直接写在策略代码注释里), 而非凭情绪临时决定;
- 有了明确规则, 砍策略时不再纠结, 加仓时也不会贪婪。

策略管理从凭感觉 => 变成按规则办事。

运行结果

TO DO: 把模拟盘当成一个虚拟策略评估

前面跑的是 6 个固定示例 (KPI 写死, 反复跑都不会变).

这里把整个模拟盘账户当成一个虚拟策略 (假名 `sim_portfolio`), 用账户的真实日收益曲线算 KPI, 看系统给什么建议.

假设模拟盘从 4 月 1 日跑到 4 月 21 日, 14 个交易日, 系统返回:

指标	实算值	门槛 (paper → probation)	是否满足
区间累计收益	+4.42%	> 0%	满足
最大回撤	1.90%	< 5%	满足
Sharpe (年化)	4.48	(看 probation → production)	--
交易日数	14	>= 20	不满足 (差 6 天)

paper 阶段维持, 暂不升 probation. 理由: 收益和回撤都达标, 但 14 天 < 20 天.

运行结果

关键洞察：时间门槛是防短期运气的关键

账户在 14 天里跑出 Sharpe 4.48 看起来完美, 但样本量太小, 这种短期数字不可靠 (业界顶级量化基金长期 Sharpe 也就 1.5~3.0, 4.48 属于运气好)

系统设的 至少 20 天 阈值就是为了拦住这种短期运气好 → 立刻 all in → 接下来被市场打脸的常见错误.

业内做法更保守: 公募基金通常要求纸交易 3 个月起步才让策略上小资金试盘.

运行结果

Sharpe 4.48 不能拿出去吹: 它有几个虚高来源:

- 样本只 14 天, 波动率被低估
- 没扣交易成本 (滑点 / 印花税), 实盘后 Sharpe 会打 5-7 折
- 区间内是平静市场, 没经历真正的波动测试

真正的用途:

跟自己历史的 Sharpe 比 -- 这周 4.48 vs 上周 3.2 → 改善了;

这周 4.48 vs 三个月平均 6.0 → 退化了, 警惕.

打卡：策略生命周期



以几个示例策略 (覆盖孵化 / 纸交易 / 试用 / 主力 / 退役) 跑一轮自动迁移为例, 让哪个该上 / 哪个该砍 / 哪个该加资金全部由 KPI 决定:

- 阶段定义: 孵化 → 纸交易 → 试用 → 主力 → 退役, 每段对应不同资金占比, 风险递减地往里加钱。
- KPI 门槛: 各阶段往上升的硬条件 (Walk-Forward 鲁棒性 / 累计收益 / 实盘 Sharpe / 滚动 Sharpe), 不达标不放行, 持续低于阈值直接退役。
- 注册中心: 把每个策略的当前阶段 + 历次 KPI 快照 + 升降级日志持久化到一个 JSON 文件, 方便 Web 直接读。
- 自动迁移: 一行命令把所有在册策略按当前 KPI 跑一遍判定, 触发的升降级写回文件并打印日志 (谁从哪升到哪 / 理由是什么)。



Thank You
Using data to solve problems

